FROM-SCRATCH PLAN

How would you design Patient Access API?

CSV files land in S3, move through Databricks, publish FHIR, and are served by a member-scoped API. Interview questions follow every step. 6 October 2026.

This is a design plan, not a security or compliance approval. Do not implement token handling, member match, or FHIR publication until that review is done. No PHI is included. External research systems were read-only. See 00-Cursor-Read-Only-Disclaimer.pdf.

Pipeline in one line

Payer CSV, then S3 inbox, checksum and quarantine, S3 raw, batch registry, Databricks bronze (Delta, declared schema), silver conform, DQ gate, gold marts, member-to-FHIR crosswalk, FHIR mapping, IG validation, private publish to the FHIR server, then a SMART authorization server and an HTTPS gateway. That gateway is the Patient Access API.

Databricks stays in the air-gapped VPC and reads S3 through a gateway endpoint. Public clients reach only the gateway, over HTTPS, with HTTP redirected to HTTPS. Writer credentials and app secrets come from the secret store and are never logged.

End-to-end shape

- **Steps 1-2.** Contract and CSV layouts. Files exist to fill FHIR.
- **Steps 3-4.** S3 landing and a batch registry. Incomplete files never reach Spark.
- **Steps 5-8.** Bronze, silver, DQ, gold on Delta. Claims merge. Directory and formulary snapshot.
- **Steps 9-12.** Crosswalk, map, validate, publish. Opaque FHIR ids. Upsert by business key.
- **Steps 13-14.** The API. One patient per token. Compartment enforced. Per-app revoke.
- **Step 15.** Metrics and rollback that do not need a clinical row to explain a failure.

Step 1. Fix the API contract before any file moves

Patient Access is a read API. A member, through an app they authorize, reads their own claims, clinical, coverage, formulary, and in-network provider data as FHIR R4 (4.0.1). The payer core stays the system of record. This pipeline publishes a FHIR projection and serves it. It does not replace adjudication.

- **In:** CMS Patient Access rule plus CARIN Blue Button, Da Vinci PDex, US Core, Drug Formulary, and Plan-Net must-support elements.
- **Out:** a resource list, a scope list, and a freshness target per domain. Claims and clinical are member-scoped. Formulary and provider directory are plan-scoped and still use the same front door.
- **Rule:** design the public contract first. CSV columns exist only to fill that contract.

Potential interview questions

1. *What does Patient Access return, and what must it refuse to return?*
2. *Why is the API FHIR R4 instead of a custom JSON API over the claims warehouse?*
3. *Which resources are patient-compartment reads, and which are plan-level (formulary, directory)?*
4. *What changes between the CMS-9115 Patient Access set and the CMS-0057 set, and how do you version both?*
5. *Who is the audience: the member app, a provider app, or another payer? Why does that split the APIs?*

Step 2. Define one CSV contract per domain

Payers deliver flat files. CSV is the only ingress format. One file family per domain, with a header row, UTF-8, a comma separator, and a published column list. Production rejects undeclared columns.

- **Member:** member_id, identifier_system, name parts, birth date, gender, address, coverage pointers. member_id is a business key, never a FHIR id.
- **Coverage:** coverage_id, member_id, plan_id, period, relationship to subscriber.
- **Claims:** claim_header and claim_line. Header holds claim_id, member_id, type, billable period, and total. Line holds line number, product or service code, modifiers, place of service, and adjudication amounts.
- **Clinical:** separate files for encounter, condition, observation, procedure, allergy, immunization, medication request, and document-reference metadata. Codes carry system and code, not display text alone.
- **Formulary:** plan, drug code, tier, prior-auth flag, quantity limit. Not keyed by member.
- **Directory:** organization, location, practitioner, practitioner role, network. A snapshot file, not a claim-style incremental.
- **Control file:** file name, row count, byte size, checksum, domain, extract time. The loader trusts the control file, then verifies it.

Potential interview questions

1. *Why insist on CSV at the edge instead of asking the payer to send FHIR?*
2. *What belongs in a header file versus a line file, and what join key do you mandate?*
3. *How do you represent a code (system, code, display) in a flat CSV without losing the system URI?*
4. *Why is a formulary file not allowed to contain member_id?*
5. *What is in the control file, and which mismatches reject the batch before Spark starts?*

Step 3. Land files in S3

The payer drops CSV into an inbox prefix. A scanner moves a complete, checksum-matched set into raw. Quarantine holds everything else. Databricks never reads the inbox.

- **Prefixes:** inbox/{domain}/, raw/{domain}/extract_date={yyyy-mm-dd}/batch_id={id}/, quarantine/{domain}/{reason}/. Partition values are dates and batch ids. Keys must not contain member ids or claim numbers.
- **Object settings:** SSE-KMS, bucket policy locked to the ingest role and the Databricks instance profile, versioning, access logging to a separate log bucket. Block public access.
- **Network:** S3 is reached through a gateway endpoint. Databricks sits in an air-gapped VPC and does not pull files over the public internet. Anything that must leave that VPC uses a bridge.
- **Arrival:** write the CSV, then write the control file last. The mover runs only when the control file exists and the checksum matches. The same batch_id written twice is a no-op if the checksum matches, and a quarantine event if it does not.
- **Logs:** log bucket, key, domain, batch_id, row count, and result. Never log a CSV row.

Potential interview questions

1. *Draw the S3 prefix layout. Why is extract_date a partition, and why is member_id forbidden in the key?*
2. *A file is re-dropped with the same name and a different checksum. What happens?*
3. *Why must the control file be written last?*
4. *Databricks is air-gapped. How does the job read S3, and what is not allowed to call the public internet?*
5. *Which S3 controls matter here (encryption, block public access, versioning, access logs), and what must those logs exclude?*

Step 4. Register the batch

A small Delta control table records every batch before any domain table is touched. Downstream tasks refuse to run without a row in status landed.

- **Columns:** batch_id, domain, extract_date, s3_prefix, row_count, checksum, status, started_at, finished_at. No payload columns.
- **Statuses:** landed, bronze_ok, silver_ok, dq_ok, gold_ok, fhir_ok, published, or failed at a named stage.
- **Idempotency:** the primary key is batch_id plus domain. A retry resumes at the last successful status.

Potential interview questions

1. *Why a batch registry instead of letting the job read whatever is in the prefix?*
2. *How does a retry know where to resume?*
3. *What do you store in the registry, and what do you refuse to store?*

Step 5. Bronze in Databricks

Bronze is the CSV, unchanged in meaning, stored as Delta. Types are declared. Nothing is cleaned here except file-level metadata columns.

- **Read:** explicit schema from a versioned contract. Schema inference is off. Bad rows go to a bronze reject table with a reason code. The raw line is not written to the application log.
- **Write:** Delta, partitioned by extract_date. Use Auto Loader or COPY INTO with a checkpoint per domain. Merge on batch_id so a rerun replaces that batch slice instead of appending a second copy.
- **Added columns:** _batch_id, _source_file, _ingest_ts. _source_file is the S3 key, which must not contain member identifiers.
- **Small files:** incoming CSVs are often tiny. Compact on write so later stages do not open thousands of files.
- **Spark:** no collect(), toPandas(), or take() over claim or clinical sets. A count is allowed only to reconcile the batch to the control-file row count.

Potential interview questions

1. *Why is schema-on-read the wrong default for this CSV?*
2. *Auto Loader versus a scheduled COPY INTO: when do you pick each?*
3. *How do you make bronze idempotent for a replayed batch?*
4. *Where do malformed CSV rows go, and why is the raw line absent from the driver log?*
5. *You have 2,000 files of a few megabytes each. What breaks, and how do you compact?*
6. *Why add _batch_id and _ingest_ts, and why is a count justified here?*

Step 6. Silver: conform the rows

Silver is the first place values change. One canonical type, one identifier treatment, one code structure. Grain stays the same as the source file.

- **Identifiers:** member_id stays a string. Never cast to integer, or leading zeros disappear. Trim, and attach identifier_system as a URI.
- **Dates:** parse a closed set of input patterns into a date type. Unparseable dates null the column and raise a DQ flag. Do not guess.
- **Codes:** split into system and code. Unknown systems are flagged, not silently mapped.
- **Claims adjustments:** a void-and-replace uses claim_id plus a version. A duplicate file is dropped using the registry checksum, not by deleting history.
- **Late data:** a claim for an older service date can arrive in today's batch. Silver accepts it. Gold merges it.
- **Columns:** drop fields the FHIR contract does not need before any wide join.

Potential interview questions

1. *Member ids arrive with leading zeros. What fails if silver casts them to bigint?*
2. *Three date formats appear in one claims file. Where do you normalize, and what is the null policy?*
3. *How do you tell a void-and-replace claim from a duplicate delivery of the same file?*
4. *A claim with an old service date lands today. Do you reject it? Why?*
5. *Why drop columns in silver, before the gold join?*

Step 7. Data-quality gate

DQ runs on silver and blocks FHIR publish. It does not block bronze, because operations still needs the landed file.

- **File rules:** row count equals the control file, required columns present, checksum already verified.
- **Blocking row rules:** missing member_id on a member-scoped file, orphan claim line, orphan header with no member, missing code system, duplicate primary key inside the batch.
- **Warnings:** missing optional address, unknown display text, formulary drug not in the reference code set.
- **Quarantine:** blocking row failures copy the key and the rule id into a DQ table. For claims, a broken header blocks that claim_id, not the whole nightly file, once file-level checks have passed.
- **Visibility:** a DQ summary (rule id, domain, batch_id, fail count) is safe to show. Row values are not written to logs.

Potential interview questions

1. *Which failures fail the batch, which fail one claim, and which only warn?*
2. *Why does DQ run after silver and not inside the CSV parser?*
3. *How do you give operations a DQ dashboard without logging member ids or dollar amounts?*
4. *Two percent of lab rows lack a code system. Quarantine the row, the file, or only that resource?*
5. *How do you keep the DQ check idempotent on retry?*

Step 8. Gold subject-area marts

Gold is the stable relational shape the FHIR mapper reads. One mart per Patient Access domain. Delta MERGE for incremental domains. Full overwrite only for snapshot domains.

- **patient_mart:** one current row per member_id. Attributes the Patient resource needs.
- **coverage_mart:** one row per coverage_id, foreign key member_id and plan_id.
- **claim_mart:** header grain plus a child line table. This is the source of ExplanationOfBenefit.
- **Clinical marts:** one table per resource family, each with a stable business key.

- **formulary_mart and directory_mart:** plan-level. Reload from the latest successful snapshot. Do not MERGE a directory the way you MERGE a claim.
- **Joins:** join on member_id and claim_id. Broadcast only a small, stable dimension such as a plan or code table. Salt a skewed plan id if one group dominates claim lines. Do not explode lines against diagnoses without a bounded array.
- **Write mode:** MERGE claims on claim_id. Overwrite the current directory snapshot and swap a view.

Potential interview questions

1. *What is the grain of claim_mart, and why is that not the same grain as the API ExplanationOfBenefit?*
2. *Why MERGE claims but overwrite provider directory?*
3. *A single plan id owns most claim lines. How do you handle skew without a cartesian join?*
4. *When is a broadcast join legal here, and when is it not?*
5. *Why are formulary and directory marts forbidden from carrying member_id?*

Step 9. Identity crosswalk

FHIR Patient.id is an opaque id minted by this platform. The payer member_id is stored on Patient.identifier with a fixed system URI. The crosswalk maps member_id to fhir_patient_id and is created on first successful publish.

- **Mint once:** MERGE into the crosswalk. An existing member_id keeps its fhir_patient_id across batches.
- **Fail closed:** the API searches Patient by identifier system and value. That search must return one row. Zero or many is an authentication failure.
- **Dependents:** each person is their own Patient. Coverage.relationship links subscriber and beneficiary. A token for the dependent does not authorize the subscriber compartment.
- **Rebuild:** the crosswalk is recoverable from the FHIR store identifier search plus gold. Do not mint a second id if the table is dropped.

Potential interview questions

1. *Why is FHIR Patient.id not equal to the payer member id?*
2. *The identifier search returns two Patients. What does login do, and why?*
3. *How can a dependent read their own explanation of benefit and not the subscriber's?*
4. *The crosswalk table is lost. How do you rebuild it without minting new ids?*
5. *What breaks in member match if silver wrote a different identifier system URI than the API searches?*

Step 10. Map marts to FHIR resources

A deterministic mapper reads gold and writes FHIR JSON. One resource builder per type. Mapping is versioned and tested, not an ad-hoc notebook.

- **Patient and Coverage** come from patient_mart and coverage_mart. Coverage.beneficiary references Patient/{fhir_patient_id}.
- **ExplanationOfBenefit** comes from claim_mart using the CARIN Blue Button profile. Header fields on the EOB, lines as items, adjudication on those items. The patient reference comes only from the crosswalk.
- **Clinical** resources use US Core and PDex: Encounter, Condition, Observation, Procedure, AllergyIntolerance, Immunization, MedicationRequest, DocumentReference. Each has a subject reference to that Patient.
- **Formulary** resources have no patient subject. **Directory** uses Plan-Net organization, location, practitioner, role, and network resources.
- **References:** resolve ids from the crosswalk and from business-key maps. Build those maps as Delta tables.

- **Bundles:** for bulk publish, write NDJSON or bounded transaction batches grouped by patient. One giant transaction bundle will time out the FHIR server.

Potential interview questions

1. *Walk one claim header and its lines into an ExplanationOfBenefit. Which profile, and what is must-support?*
2. *How do you build Coverage.beneficiary without joining every line to every patient?*
3. *Transaction bundle versus batch bundle versus NDJSON for a full claims load. What do you pick and why?*
4. *Why can a formulary resource omit subject, and how does the API still protect it?*
5. *DocumentReference points at a document. Where does the binary live, and what must the API keep out of logs?*

Step 11. Validate against the implementation guide

Validation is a gate before publish. Use the HL7 validator with the pinned IG package for the rule version you are certifying. Do not validate with a Python UDF row by row across the full claim set.

- **How:** write resource files to a volume and run the validator as a batch job, one process per file partition, not per row on the driver. Record error counts by resource type and severity.
- **Block:** fatal and error severities stop publish for that resource. Warnings are stored. Informational issues do not block.
- **Versions:** CMS-9115 and CMS-0057 packages are different pins. A batch declares which pin it targets.
- **Output:** a validation summary (batch_id, resource type, severity, issue code, count). Do not log OperationOutcome text that echoes a resource body.

Potential interview questions

1. *Why is a Python UDF that calls the validator per row the wrong design on a full claims history?*
2. *Which validator severities block publish?*
3. *How do you certify 9115 and 0057 on the same platform without mixing profiles in one batch?*
4. *What do you persist from a failed validation, and what do you refuse to persist?*

Step 12. Publish to the FHIR server

The FHIR server is the runtime copy. Spark talks to it on the private network with a server credential from the secret store. Apps never use that credential.

- **Upsert:** conditional update on the business identifier, using the crosswalk id when the resource already exists. A rerun updates. It does not insert a second ExplanationOfBenefit for the same claim_id.
- **Throttle:** cap in-flight writes. A Spark stage that opens one HTTP call per row with default parallelism will overload the server.
- **Voids:** a voided claim updates EOB status or deletes, per the IG. A whole-patient hard delete is a ticketed job, not a nightly side effect. Keep tombstones and backups long enough to restore a bad publish.
- **Snapshots:** directory and formulary publish as a new version. The API reads the active version only after the load commits.
- **Secrets:** the writer credential lives in the workspace secret scope. If infrastructure code creates that secret, set recovery window to 0. The job fails immediately when the secret is missing. Never print it.

Potential interview questions

1. *How do you upsert an ExplanationOfBenefit so a replay does not duplicate it?*
2. *What happens to the FHIR server if each Spark task writes one resource with no concurrency cap?*
3. *A claim is voided. Do you delete the resource or update status, and who decides?*
4. *How do you roll back a bad nightly publish?*
5. *Where does the writer credential live, and what must the job do if it is missing?*

Step 13. Patient Access API front door

Two processes sit in front of the store. An authorization server implements SMART on FHIR for member standalone launch. A gateway is the only public FHIR endpoint. The store is not on the internet.

- **Launch:** the app redirects the member to authorize. The member signs in at the payer identity provider. The authorization server reads a member-id claim, searches Patient by identifier, and issues a bearer token whose patient context is that one fhir_patient_id. Scopes are patient-level reads for the resource types you publish.
- **Gateway:** validates the token, enforces the patient compartment on every search and read, and proxies to the FHIR server. Public listeners are HTTPS only. HTTP redirects to HTTPS and does not serve API bodies.
- **Paths:** separate capability bases for claims, clinical, formulary, and directory. Clients discover the contract at metadata, not from a homemade OpenAPI file.
- **Compartment:** a token for patient A cannot read patient B, including by following links or references that leave the compartment. An ambiguous match is an authorization error.
- **Errors:** 401 unauthenticated, 403 outside scope or compartment, 404 unknown id inside the compartment, 429 rate limited. The OperationOutcome does not echo the token or another patient's data.
- **Rate limit:** per application client id, at the gateway, before the store.

Potential interview questions

1. *Draw the call from a third-party app to one ExplanationOfBenefit. Where are the member credentials checked?*
2. *Why standalone patient launch, and not a client-credentials system scope, for this API?*
3. *How do you stop a token for one member from walking references to another member?*
4. *What is the difference between 401, 403, 404, and 429 here, and what must the body exclude?*
5. *Why is CapabilityStatement the contract instead of an OpenAPI document?*
6. *Why is the FHIR server invisible to the application, and what credential is the app allowed to hold?*

Step 14. App registration, consent, and revoke

The payer admin approves which apps may call the API. The member approves which of those apps may see their data, and can revoke one app.

- **App registry:** client id, redirect URIs, scopes requested, status. Created when an admin approves a registration. Confidential clients keep a secret in the secret store.
- **Grant:** created only when the member finishes the authorize redirect. Stored as member subject, client id, scopes, and fhir_patient_id. Not a copy of the clinical record.
- **Revoke:** a member privacy page lists grants and deletes one. The gateway rejects that client's tokens for that patient on the next call. Other apps keep working.
- **Opt-out:** a plan-level opt-out is a different record from an OAuth grant. If the member opts out, the gateway returns 403 for every app. Do not delete the FHIR store to implement opt-out.

Potential interview questions

1. *Who registers an app, and who consents to it? Why are those different people?*
2. *How does revoking one app leave the member's other apps working?*
3. *What is stored in a grant row, and what is not?*
4. *How is a plan opt-out different from OAuth consent?*
5. *What remains broken if logout does not drop the bearer token the app is holding?*

Step 15. Audit, metrics, and operations

CMS expects evidence of member use. Engineering expects evidence of pipeline health. Those are different tables. Both are free of row-level PHI.

- **Pipeline metrics:** per batch and domain: row counts, DQ fail counts, resources validated, resources published, duration, bytes read. Alert if published count drops to zero or the DQ fail rate jumps.
- **API metrics:** unique members with a successful login in the window, members with more than one login, and request counts by door and status. Login numbers come from the authorization server. Request counts come from the gateway. One written definition so the two reports cannot disagree.
- **Audit:** timestamp, client id, route template, status, latency. No identifier in the query string, no response body, no token.
- **Promotion:** dev, then stage, then prod, same job code. Dev CSVs are synthetic. Prod jobs read only the prod raw prefix.
- **Rollback:** registry status, Delta time travel on gold, and a republish of the last good batch. FHIR upserts make republish safe.

Potential interview questions

1. *Which system is the source of truth for unique members who used Patient Access, and why must the warehouse report use that same definition?*
2. *What is safe to put in an API access log?*
3. *A bad claim file was published. Walk the rollback using the registry, Delta, and the FHIR upsert.*
4. *How do you prove a rerun is safe in production?*
5. *What synthetic-data rule applies in dev, and how do you keep prod prefixes out of a dev job?*

What a strong walkthrough sounds like

Start from a nightly claims CSV. Show the control file, the raw prefix, the bronze schema, the silver date and identifier rules, the DQ rule that blocks an orphan line, the gold MERGE on claim_id, the crosswalk lookup, the CARIN ExplanationOfBenefit, the validator gate, the conditional update, and the gateway check that the token patient matches the EOB patient. Then say what happens if the same file arrives twice, and what happens if identifier search returns two Patients.

Potential interview questions

1. *Whiteboard the path from a claims CSV row to the JSON a member app receives. Where can PHI leak, and what control stops each leak?*
2. *Which stages are idempotent, and which single stage, if skipped, duplicates a claim in the API?*
3. *What would you cut from v1, and what would you refuse to cut (compartment check, DQ block, checksum)?*